COMP 2233: Data Structures

Chapter 6: Pointers in C and C++

Assoc.Prof.Dr. Ömer ÇETİN

Problem Solving with Computer

Problem Solving Pipeline

1. Problem Definition: Real-world domain statement.
2. Algorithm Development (Human): Step-by-step logic design.
3. Program Development (Human): Writing C/C++ source code.
4. Program Translation (Software): Compilation to machine instructions.
5. Program Execution (Hardware): Executing instructions in RAM & CPU.

Role of Pointers in Pipeline

- Hardware Execution Link: Pointers interact directly with physical RAM addresses during Program Execution.
- Memory Efficiency: Enables dynamic allocation during runtime rather than compile-time static bounds.
- Error Loops:
  - Syntax Errors: Caught during Program Translation by compiler.
  - Semantic Errors: Logical bugs & invalid memory access caught during Execution.

Course Syllabus & Chapter Map

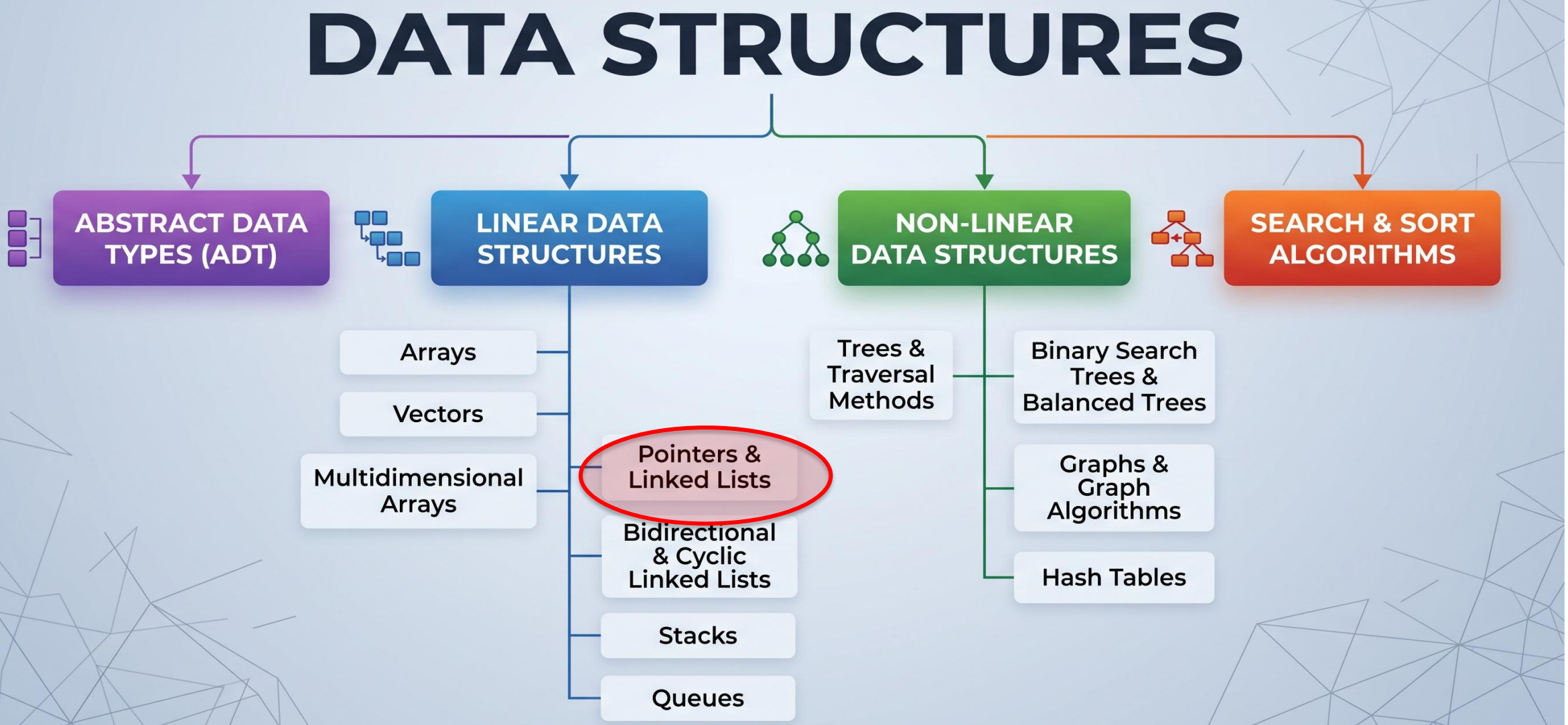
First-Half Topics

1. Introduction to Data Structures
2. Algorithm Analysis & Complexity
3. Data, Data Structures & Data Models
4. Single Dimension Arrays & Vectors
5. Multi-Dimension Arrays
-> 6. Pointers (CURRENT CHAPTER)
7. Linked Lists

Second-Half Topics

8. Doubly Linked Lists
9. Circular Linked Lists
10. Queues & Stacks
11. Trees & Binary Search Trees
12. Graphs & Network Models
13. Hash Tables
14. Search and Sort Algorithms

Course Content



Introduction to Pointers & Core Concepts

1. What is a Pointer?

- **Definition**: A variable that stores the physical memory address of another variable.
- **Indirect Access**: Instead of holding data values directly, it holds the location where data resides.
- **Low-Level Control**: Core feature of C and C++ for direct hardware interaction.

2. Memory Address

- **RAM Slots**: Every variable declared in a program is allocated a unique memory address.
- **Address-Of Operator (&)**: Used to retrieve the physical address of a variable (e.g., &var).
- **Hexadecimal Notation:** Addresses are represented as hex numbers (e.g., 0x7ffc29dca8b8).

3. Dereferencing

- **Indirection Operator (*):** Used to access or modify the value stored at the address.
- **Reading Value:** int x = *ptr; reads memory location.
- **Writing Value:** *ptr = 20; mutates the memory location directly.

Why Pointers are Important in Data Structures

Dynamic Allocation

- **Runtime Sizing:** Memory is allocated dynamically during program execution using Heap RAM.
- **Eliminates Waste:** Avoids over-estimating static array sizes at compile time.
- **Flexible Buffers:** Allocation grows or shrinks based on actual problem inputs.

Structure Linking

- **Non-Contiguous Storage:** Data items can be scattered anywhere across RAM.
- **Pointer Chaining:** Nodes are connected via next/previous memory pointers.
- **Essential Structures:** Linked Lists, Binary Trees, Graphs, and Hash Tables rely 100% on pointers.

Program Optimization

- **Efficient Pass-by-Reference:** Passing large structures via pointer avoids copying kilobytes of data.
- **Fast Re-linking:** Swapping node pointers in O(1) time avoids costly O(N) array element shifting.
- **Direct Hardware Access:** Enables building high-performance OS kernels & device drivers.

Pointer Syntax & Mechanics in C/C++

3 Core Syntax Steps

1. Declaration:

int *ptr; // Declares a pointer variable pointing to an integer.

2. Assigning Memory Address:

int a = 10;

ptr = &a; // ptr now holds physical address of 'a'.

3. Dereferencing:

int val = *ptr; // Reads value at address stored in ptr (val = 10).

*ptr = 25; // Modifies 'a' directly to 25.

Syntax Demonstration Code

```
// C/C++ Pointer Mechanics Example
#include <stdio.h>

int main(void) {
    int a = 10;
    int *ptr = &a; // Declare & assign

    printf("Value of a: %d\n", a);      // 10
    printf("Address of a: %p\n", &a);   // 0x7ff...
    printf("Value of ptr: %p\n", ptr);   // 0x7ff...
    printf("Dereferenced: %d\n", *ptr); // 10

    *ptr = 25; // Mutate value via pointer
    printf("New value of a: %d\n", a);  // 25
    return 0;
}
```

Pointer Types (1/2): Null & Dangling Pointers

a) Null Pointer

- **Definition:** A pointer that explicitly points to NO valid memory address (0 / NULL / nullptr).
- **Purpose:** Safe initialization state before allocating real memory.
- **C Syntax:** int *ptr = NULL;
- **C++ Syntax:** int *ptr = nullptr;
- **Safety Guard:** Always check 'if (ptr != NULL)' before dereferencing!

b) Dangling Pointer

- **Definition:** A pointer that still stores the memory address of a buffer that HAS ALREADY BEEN FREED.
- **Hazard:** Accessing a dangling pointer causes undefined behavior or memory corruption.
- **Example Code:**

```
int *ptr = (int *)malloc(sizeof(int));
free(ptr); // Memory deallocated!
// ptr is now a Dangling Pointer!
```

- **Prevention:** Set 'ptr = NULL;' immediately after freeing memory!

Pointer Types (2/2): Void & Double Pointers

c) Void Pointer (Generic)

- **Definition:** A generic pointer (void*) that holds the address of ANY data type.
- **Type Agnostic:** Used heavily in C dynamic memory functions like malloc().
- **Casting Rule:** Cannot be dereferenced directly without explicit type casting.
- **Code Example:**

```
void *ptr;
int a = 5;
ptr = &a;
printf("%d", *(int*)ptr); // Cast required!
```

d) Pointer to Pointer (Double)

- **Definition:** A pointer variable (int**) that stores the memory address of ANOTHER pointer.
- **Indirection Level:** Double indirection (**pptr).
- **Application:** Used in 2D dynamic matrices & modifying head pointers inside C functions.
- **Code Example:**

```
int a = 10;
int *ptr = &a;   // Holds address of 'a'
int **pptr = &ptr; // Holds address of 'ptr'
```

Pointers and Arrays: Address Offset Arithmetic

Array Name as Base Pointer

- **Array Name Decay:** The name of an array acts as a constant pointer to its first element (index 0).
- **Base Assignment:** int arr[5] = {1, 2, 3, 4, 5}; int *ptr = arr; (Equivalent to &arr[0]).
- **Equivalent Notations:**

  arr[i] <==> *(arr + i)

  &arr[i] <==> (arr + i)

- **Offset Calculation:** Pointer arithmetic automatically scales by sizeof(type) bytes!

Array Offset Code Demonstration

```
// Pointers and Array Traversing
#include <stdio.h>

int main(void) {
    int arr[5] = {10, 20, 30, 40, 50};
    int *ptr = arr; // Points to arr[0]

    printf("First element: %d\n", *ptr);        // 10
    printf("Second element: %d\n", *(ptr + 1)); //
20

    ptr++; // Pointer arithmetic: move 4 bytes
forward
    printf("After ptr++: %d\n", *ptr);          //
20
    return 0;
}
```

Pointers in Dynamic Data Structures

a) Linked Lists

- Self-referential nodes connected sequentially via 'next' pointers.
- Allows O(1) node insertion/deletion without array element shifting.

b) Binary Trees

- Hierarchical node structures maintaining 'left' and 'right' child pointers.
- Enables O(log N) logarithmic search, insertion, and deletion.

c) Graphs

- Complex network graphs implemented via pointer adjacency lists.
- Models real-world networks, social graphs, and routing paths.

d) Dynamic Resizable Arrays

- Heap buffers dynamically resized at runtime via pointers.
- Powers C++ std::vector and dynamic array reallocation.

Memory Management: Dynamic Allocation & Freeing

Dynamic Memory Allocation Functions

- **Heap RAM Allocation:** Memory requested during execution lives on the Heap.
- **C Functions:**
  - malloc(size): Allocates raw uninitialized memory block.
  - calloc(num, size): Allocates zero-initialized memory block.
  - realloc(ptr, new_size): Resizes previously allocated block.
  - free(ptr): Deallocates memory block back to OS Heap.
- **C Syntax Example**:

```
int *ptr = (int *)malloc(10 * sizeof(int));
free(ptr);
```

Avoiding Memory Leaks

- **What is a Memory Leak?** Occurs when dynamically allocated Heap RAM is no longer referenced but never deallocated via free() or delete.
- **Consequences:** System consumes increasing memory over time, degrading performance or causing crashes.
- **Gold Rule of Memory:**

  Every malloc() MUST have a corresponding free()!

  Every new MUST have a corresponding delete!
- **Post-Free Reset:** Always set 'ptr = NULL;' after deallocating.

Common Pointer Errors & Best Practices

1. Null Pointer Dereferencing

- **Cause:** Dereferencing a pointer that contains NULL / nullptr.
- **Result:** Immediate Runtime Segmentation Fault (Crash!).
- **Fix:** Guard with 'if (ptr != NULL)' check.

2. Memory Leaks

- **Cause:** Allocating Heap RAM and losing the pointer reference without calling free().
- **Result:** Progressive RAM exhaustion.
- **Fix:** Track allocations & free before scope exit.

3. Dangling Pointers

- **Cause:** Using a pointer after its target memory block has been deallocated.
- **Result:** Undefined behavior & memory corruption.
- **Fix:** Immediately assign 'ptr = NULL;' after free().

4. Invalid Pointer Arithmetic

- **Cause:** Incrementing/decrementing pointers beyond valid buffer bounds.
- **Result:** Overwriting unintended variables in RAM.
- **Fix:** Keep strict boundary size counters.

Pointers in High-Level Languages (Python & Java)

Direct Pointers (C / C++)

- **Explicit Memory Control:** Programmers directly manipulate raw physical addresses.
- **Manual Management:** Mandatory manual dynamic allocation & deallocation.
- **High Risk & Speed:** Maximum execution speed, but susceptible to memory corruption, dangling pointers, and leaks.
- **Hardware Access:** Ideal for system programming, graphics engines, and embedded systems.

Object References (Python & Java)

- **Automatic Management:** High-level languages hide raw addresses behind Object References.
- **Garbage Collection (GC):** Background process automatically deallocates unreferenced objects.
- **Safe Execution:** Prevents dangling pointers, buffer overflows, and explicit segfaults.
- **Abstraction Trade-off:** Convenience and safety come at the cost of GC runtime pauses and memory overhead.

C++ Pointer Operation Examples Overview

Basic Operations (1 - 4)

1. **Declaring and Initializing Pointers:**
   Demonstrates &, *, and pointer address assignment.
2. **Pointer Arithmetic:**
   Demonstrates ptr++ and ptr-- across arrays.
3. **Pointer to Pointer:**
   Demonstrates double indirection (**pptr).
4. **Passing Pointers to Functions:**
   Demonstrates pass-by-reference mutation.

Advanced Operations (5 - 8)

5. **Dynamic Memory Allocation:**
   Demonstrates C++ 'new', 'delete', and 'delete[]'.
6. **Null Pointers:**
   Demonstrates C++11 'nullptr' safety guards.
7. **Pointer to a Constant:**
   Demonstrates 'const int *ptr' (read-only data).
8. **Constant Pointer:**
   Demonstrates 'int *const ptr' (fixed address).

Example 1: Declaring & Initializing Pointers

Source Code

```
// C++ Example 1: Declaration & Dereferencing
#include <iostream>
using namespace std;

int main() {
    int var = 10;    // Normal integer variable
    int *ptr = &var; // Pointer initialized to address of
'var'

    cout << "Value of var: " << var << endl;
    cout << "Address of var: " << &var << endl;
    cout << "Value stored in ptr: " << ptr << endl;
    cout << "Dereferencing ptr: " << *ptr << endl;
    return 0;
}
```

Execution Output & Trace

Terminal Output:

Value of var: 10

Address of var: 0x7ffc29dca8b8

Value stored in ptr: 0x7ffc29dca8b8

Dereferencing ptr: 10

Trace Insights:

- &var and ptr produce the EXACT same memory address.
- *ptr evaluates to 10 (the value stored inside var).

Example 2: Pointer Arithmetic across Arrays

Source Code

```
// C++ Example 2: Pointer Arithmetic
#include <iostream>
using namespace std;

int main() {
    int arr[3] = {10, 20, 30};
    int *ptr = arr; // Pointer to first element (arr[0])

    cout << "First element: " << *ptr << endl;
    ptr++; // Increment pointer to next int element
    cout << "After ptr++: " << *ptr << endl;
    ptr--; // Decrement pointer back to first element
    cout << "After ptr--: " << *ptr << endl;
    return 0;
}
```

Execution Output & Trace

Terminal Output:

Pointer pointing to first element: 10

Pointer after increment: 20

Pointer after decrement: 10

Trace Insights:

- ptr++ increments the address by sizeof(int) = 4 bytes.
- ptr now points directly to arr[1] (value 20).
- ptr-- decrements the address by 4 bytes, returning to arr[0].

Example 3: Pointer to Pointer (Double Indirection)

Source Code

```
// C++ Example 3: Double Pointer
#include <iostream>
using namespace std;

int main() {
    int var = 10;
    int *ptr = &var;   // Pointer to integer
    int **pptr = &ptr; // Pointer to pointer

    cout << "Value of var: " << var << endl;       // 10
    cout << "Value via *ptr: " << *ptr << endl;     // 10
    cout << "Value via **pptr: " << **pptr << endl; // 10
    return 0;
}
```

Execution Output & Trace

Terminal Output:

```
--------------------------------------
Value of var: 10
Value via *ptr: 10
Value via **pptr: 10
--------------------------------------
```

Trace Insights:

- pptr stores memory address of ptr.
- *pptr dereferences once -> yields address of var (ptr).
- **pptr dereferences twice -> yields actual value 10.

Example 4: Passing Pointers to Functions

Source Code

```
// C++ Example 4: Pass-by-Reference via Pointer
#include <iostream>
using namespace std;

void increment(int *p) {
    (*p)++; // Dereference and increment value
}

int main() {
    int num = 5;
    cout << "Before increment: " << num << endl; // 5
    increment(&num); // Pass address of num
    cout << "After increment: " << num << endl;  // 6
    return 0;
}
```

Execution Output & Trace

Terminal Output:

Before increment: 5

After increment: 6

Trace Insights:

- Address &num is passed to function parameter int *p.
- (*p)++ directly modifies physical memory slot of 'num'.
- Changes persist after increment() function returns!

Example 5: Dynamic Memory (`new` & `delete`)

Source Code

```
// C++ Example 5: Dynamic Heap Allocation
#include <iostream>
using namespace std;

int main() {
    // Single Integer Allocation
    int *ptr = new int;
    *ptr = 100;
    cout << "Dynamically allocated value: " << *ptr <<
endl;
    delete ptr; // Free single item

    // Dynamic Array Allocation
    int *arr = new int[5];
    for (int i = 0; i < 5; i++) arr[i] = i + 1;
    delete[] arr; // Free dynamic array!
    return 0;
}
```

Execution Output & Trace

Terminal Output:

Dynamically allocated value: 100

arr[0] = 1

arr[1] = 2

arr[2] = 3

arr[3] = 4

arr[4] = 5

Trace Insights:

- 'new int' allocates Heap memory for 1 integer.
- 'delete ptr' frees Heap RAM back to OS.
- 'new int[5]' allocates dynamic array; 'delete[] arr' frees array block!

Example 6: Null Pointers (`nullptr` in C++)

Source Code

```
// C++ Example 6: nullptr Safety Guards
#include <iostream>
using namespace std;

int main() {
    int *ptr = nullptr; // Safe initial state

    if (ptr == nullptr) {
        cout << "Pointer is null" << endl;
    }

    ptr = new int(20); // Allocate & initialize
    cout << "Allocated value: " << *ptr << endl;

    delete ptr;    // Deallocate Heap RAM
    ptr = nullptr; // Reset to nullptr for safety!
    return 0;
}
```

Execution Output & Trace

Terminal Output:

Pointer is null

Pointer value after allocation: 20

Trace Insights:

- nullptr represents type-safe null pointer in modern C++11.
- 'if (ptr == nullptr)' guards against crash.
- Resetting 'ptr = nullptr' after delete prevents Dangling Pointer bugs!

Example 7: Pointer to a Constant (`const int *ptr`)

Source Code

```
// C++ Example 7: Pointer to Constant
#include <iostream>
using namespace std;

int main() {
    const int var = 10;    // Constant variable
    const int *ptr = &var;// Pointer to constant int

    cout << "Value pointed to by ptr: " << *ptr << endl;

    // *ptr = 20; // ERROR: Cannot modify read-only data!
    int b = 30;
    ptr = &b;      // ALLOWED: Pointer itself CAN change
address!
    return 0;
}
```

Execution Output & Trace

Terminal Output:

Value pointed to by ptr: 10

Key Rules for 'const int *ptr':

- READ-ONLY DATA: Cannot mutate underlying value via '*ptr = 20;' (compiler error).
- MUTABLE ADDRESS: Pointer ITSELF can be updated to point to a different variable ('ptr = &b;').

Example 8: Constant Pointer (`int *const ptr`)

Source Code

```
// C++ Example 8: Constant Pointer
#include <iostream>
using namespace std;

int main() {
    int var1 = 10;
    int var2 = 20;

    int *const ptr = &var1; // Constant pointer to var1
    *ptr = 15;              // ALLOWED: Data CAN be
modified!
    cout << "Modified var1: " << var1 << endl; // 15

    // ptr = &var2; // ERROR: Cannot change fixed address!
    return 0;
}
```

Execution Output & Trace

Terminal Output:

Modified var1: 15

Key Rules for 'int *const ptr':

- FIXED ADDRESS: Pointer address is constant; cannot point to another variable ('ptr = &var2;' fails).
- MUTABLE DATA: Underlying data CAN be modified via dereferencing ('*ptr = 15;' succeeds).

Summary of Pointer Operations in C++

Core Practical Takeaways

- **Direct Memory Access:** Pointers provide full control over hardware RAM addresses.
- **Address-Of (&) vs Dereference (*):** & extracts RAM address, * accesses or mutates target data.
- **Array Offset Arithmetic:** ptr++ increments memory location by sizeof(data_type) bytes.
- **Pass-by-Reference:** Passing pointers to functions allows permanent variable mutation without memory copying.

Memory Safety Discipline

- **Dynamic Lifecycle:** Pair every 'new' with 'delete', and every 'new[]' with 'delete[]'.
- **Nullptr Guards:** Initialize pointers to 'nullptr' and check validity before dereferencing.
- **Dangling Pointer Prevention:** Immediately set 'ptr = nullptr;' after freeing dynamic RAM.
- **Const Qualifiers:** Use 'const int*' for read-only data and 'int* const' for fixed addresses.

Conclusion & Transition to Linked Lists

Summary of Chapter 6

- **Essential Foundation:** Mastering pointers is mandatory for understanding advanced data structures.
- **Dynamic Flexibility:** Pointers eliminate static compile-time memory limits, supporting dynamic runtime growth.
- **Efficiency Boost:** Pass-by-reference and constant-time pointer re-linking optimize execution speed.
- **Hardware Mastery:** Direct memory control forms the backbone of operating systems and algorithms.

Bridge to Chapter 7: Linked Lists

- **Next Chapter Preview:** We will use pointers to build our first fully dynamic data structure — Linked Lists!
- **Pointer Chaining:** Individual node structures containing data + next pointer.
- **Non-Contiguous Allocation:** Scattered RAM nodes linked together seamlessly.
- **Upcoming Topics:** Singly, Doubly, and Circular Linked Lists.